Degree-3 spectral refinement: public research draft and verification package

Repository source: README.md

Contents

Release: v0.1.0 (September 6, 2026)

Author: Bruce J. Swihart

Status: AI-assisted, non-peer-reviewed research draft seeking independent mathematical verification
Canonical repository: github.com/swihart/minvol-degree3-spectral-bound
AI system disclosed: OpenAI ChatGPT (GPT-5.6 Sol Pro), accessed August-September 2026

This repository is the public research and verification package for version `v0.1.0` of a proposed refinement of Nishioka's lower bound for the three-dimensional Blaschke–Lebesgue problem. The package is designed to make the argument, exact algebra, numerical cross-checks, and verification boundary easy to inspect. It is not peer reviewed or independently verified.

Candidate coefficient

The proposed bound is

$$\mathrm{Vol}(K) \geq \frac{\pi}{1914}\left(259 - 27\sqrt{\frac{15183}{17303}}\right) d^3 \approx 0.3836027047 d^3.$$

Nishioka's published coefficient is `4*pi/33 ~= 0.3807991095`. The candidate's possible contribution is the independent degree-3 spectral mechanism rather than a numerically leading coefficient.

Related recent work

A recent public HYRA manuscript, *A Certified Geometric Lower Bound for the Three-Dimensional Blaschke–Lebesgue Problem*, claims substantially stronger analytic and computer-assisted lower bounds by a different geometric method. Those claims have not been independently verified in this project. The contribution proposed here is instead a degree-3 spectral refinement of Nishioka's argument.

How to review this claim

A first-time reviewer can follow this path:

1. Read the main research draft for the integrated statement and proof.
2. Audit the novel algebra in the expanded degree-3 tensor derivation (editable Markdown).
3. Audit the route from the determinant estimate to the volume bound in the expanded bridge lemmas (editable Markdown).
4. Use the claim-dependency graph (editable Markdown) to locate the source or proof for each major step.
5. Consult the proof ledger, verification-status statement, and internal proof audit to see what has and has not been checked.
6. Follow the reproducibility guide to rerun the exact Python checks, independent R checks, and document builds.

The proof strategy is to isolate the degree-3 curvature energy, control the determinant of its curvature tensor, convert that control to a nuclear-norm estimate, use operator/nuclear duality to cap the degree-3 budget, and charge all remaining harmonic energy at the smaller coefficient `1/58`.

What has been checked

- A general degree-3 spherical harmonic is represented by a seven-parameter symmetric trace-free tensor.
- The central norm and determinant identities are integrated exactly over `S^2` with SymPy rational arithmetic.
- The crucial constants are derived from polynomial identities rather than inserted as final answers.
- A second exact formulation checks the determinant identity independently.
- Deterministic numerical quadrature and random tensor tests check conventions.
- Independent Python and base-R numerical implementations both pass; see `expected/CROSS_LANGUAGE_COMPARISON.md`.
- The final coefficient is simplified and evaluated exactly.
- The noncomputational bridge from the determinant estimate to the candidate volume bound has a line-by-line draft in `proof/BRIDGE_LEMMAS.md`.
- The degree-3 tensor identity now has a human-readable derivation in `proof/DEGREE3_TENSOR_IDENTITY.md`, including a contraction-graph count for all six quartic moments.
- The contraction-graph table is independently enumerated by exact Python code; a matching base-R script is included for cross-language checking.
- An internal line-by-line audit is recorded in `proof/PROOF_AUDIT.md`.
- The complete proof has been merged into a LaTeX manuscript in `paper/degree3_spectral_refinement.tex`; the tracked PDF was built and visually inspected in this project environment.
- The named author completed a structured understanding review, approved the revised manuscript and supporting package, and authorized preparation of the public `v0.1.0` release candidate.
- Release version, date, repository URLs, citation metadata, and public-facing status language are fixed and checked automatically.
- An initial fresh clone of commit `96cea57275070f21babf30477ab1b128ec0f5eb6` reproduced every check and build and left the tracked working tree byte-clean; see `CLEAN_CLONE_CHECK.md`. The exact `v0.1.0` release-candidate commit is tested again before tagging.

Release and verification status

Version `v0.1.0` is the initial public research draft. Before tagging, the exact release-candidate commit must pass all three hosted GitHub Actions jobs on `main` and a final clean-clone reproduction. That same commit is then tagged `v0.1.0`; the tag-triggered workflow must also pass before the GitHub release is published. The release page records the tested commit and final result.

The internal author-understanding review, manuscript review, proof audit, metadata preparation, and reproducibility work are complete for this release candidate. The principal outstanding scientific gate is review of the complete proof by an independent subject-matter expert. Peer review has not occurred.

See the release notes, `proof/PROOF_LEDGER.md`, `proof/CLAIM_DEPENDENCIES.md`, `proof/PROOF_AUDIT.md`, `proof/BRIDGE_LEMMAS.md`, and `proof/DEGREE3_TENSOR_IDENTITY.md`.

Automated verification and reproducibility

The workflow in `.github/workflows/verification.yml` runs on pushes to `main`, version tags, pull requests, and manual dispatch. It uses separate Ubuntu jobs to:

- install the declared Python dependencies and run every exact and numerical Python check;

- install R and run the independent pairing-count and numerical checks; and
- install Pandoc and TeX, rebuild the research paper and every Markdown-derived PDF, and preflight the generated PDF files and checksum manifest.

A green workflow run means that the posted code and document sources execute successfully in a fresh hosted environment. It does **not** constitute peer review or independent verification of the mathematical argument. The document job also checks consistency of the author name, AI-system identification, citation metadata, and licensing declarations.

Typeset PDF versions of the Markdown documentation

GitHub does not consistently render the repository's single-backslash LaTeX math delimiters. Typeset PDF counterparts are tracked under `rendered/markdown/`, with the source directory structure preserved. The Markdown files are the editable source files; the PDFs are rendered counterparts provided for convenient reading.

- Repository overview PDF
- Release notes PDF
- AI assistance and provenance PDF
- Recorded clean-clone verification PDF
- Reproducibility guide PDF
- Verification status PDF
- Cross-language comparison PDF
- Paper build notes PDF
- Bridge lemmas PDF
- Claim dependencies PDF
- Degree-3 tensor identity PDF
- Internal proof audit PDF
- Proof ledger PDF

Regenerate every Markdown-derived PDF from the repository root with:

```
./build_markdown_pdfs.sh
```

The build requires Pandoc, Python 3, and a XeLaTeX installation. The current pipeline has passed with Pandoc 2.19.2 locally and Pandoc 3.10.1 in GitHub Actions. The script recognizes the `\(...\)` and `\[...\]` delimiters used in the proof notes, converts the few multi-line tagged displays to valid `amsmath` environments, normalizes the PDF trailer identifiers for reproducible output, and writes a SHA-256 manifest to `rendered/markdown/SHA256SUMS.txt`.

Build the paper

The manuscript source and tracked PDF are in `paper/`. Build from the repository root with:

```
./build_paper.sh
```

The build requires `latexmk` and a standard LaTeX installation. See `paper/README.md`.

Run the exact Python checks

From the repository root:

```
python -m pip install -r requirements.txt
./run_python_checks.sh
```

The exact symbolic scripts are:

- `python/verify_degree3_identity.py`
- `python/verify_degree3_identity_alt.py`
- `python/verify_bound_arithmetic.py`
- `python/verify_pairing_counts.py`

The numerical stress test is:

- `python/numerical_stress_tests.py`

Run the R cross-check

The R scripts use base R only:

```
Rscript R/verify_pairing_counts.R
Rscript R/numerical_stress_tests.R
```

The first script independently checks the finite pairing-count table; the second is an independent numerical tensor cross-check. The general exact symbolic integration is performed by the Python/SymPy scripts.

Current verification language

Use:

> AI-assisted, non-peer-reviewed research draft seeking independent mathematical verification.

Do not use:

> Peer-reviewed, certified, or independently verified theorem.